



Université Mohammed V
Faculté des Sciences
Rabat

LIVRABLE FINAL

Dépôt Git structuré & Hébergement en ligne

Nom du Projet : Learnify

Membre	Rôle Scrum
ALOUANI Mouhcine	Product Owner
FARIZ Adam	Développeur
CHAMSSI Imane	Développeur

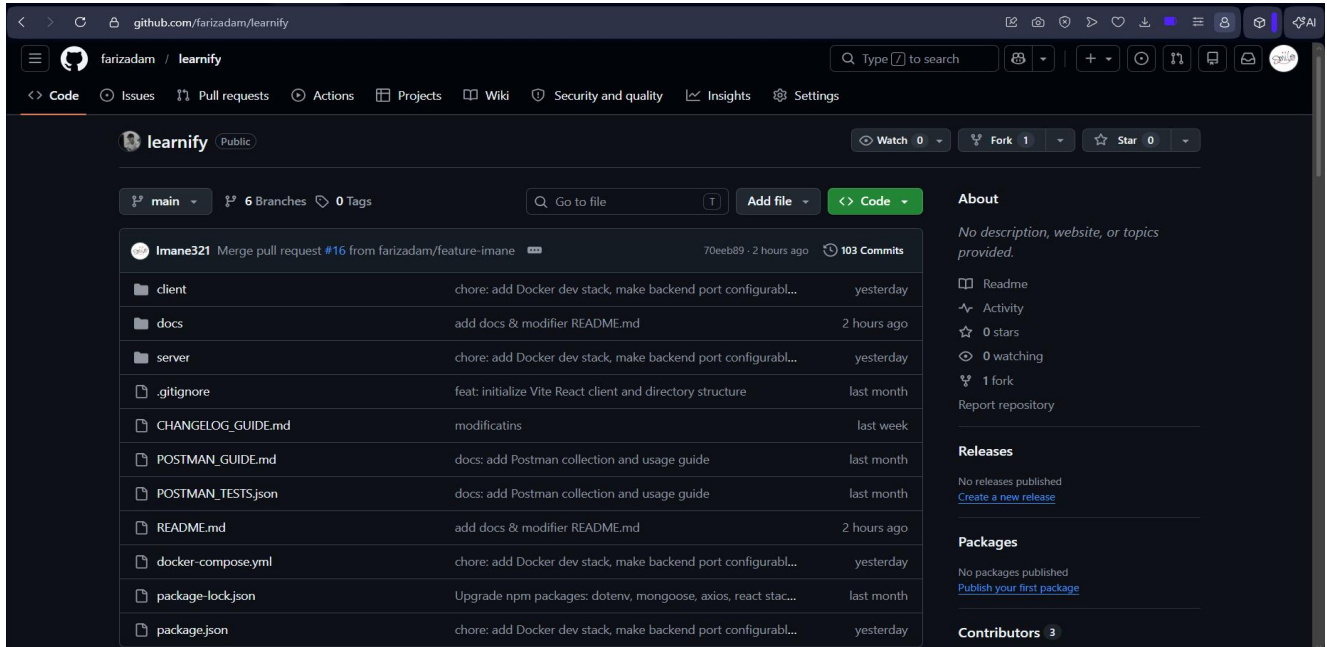
GitHub	https://github.com/farizadam/learnify
URL Déploiement	https://learnnifyyy.netlify.app/
Date	Mai 2026

Partie 1 — Dépôt GitHub structuré

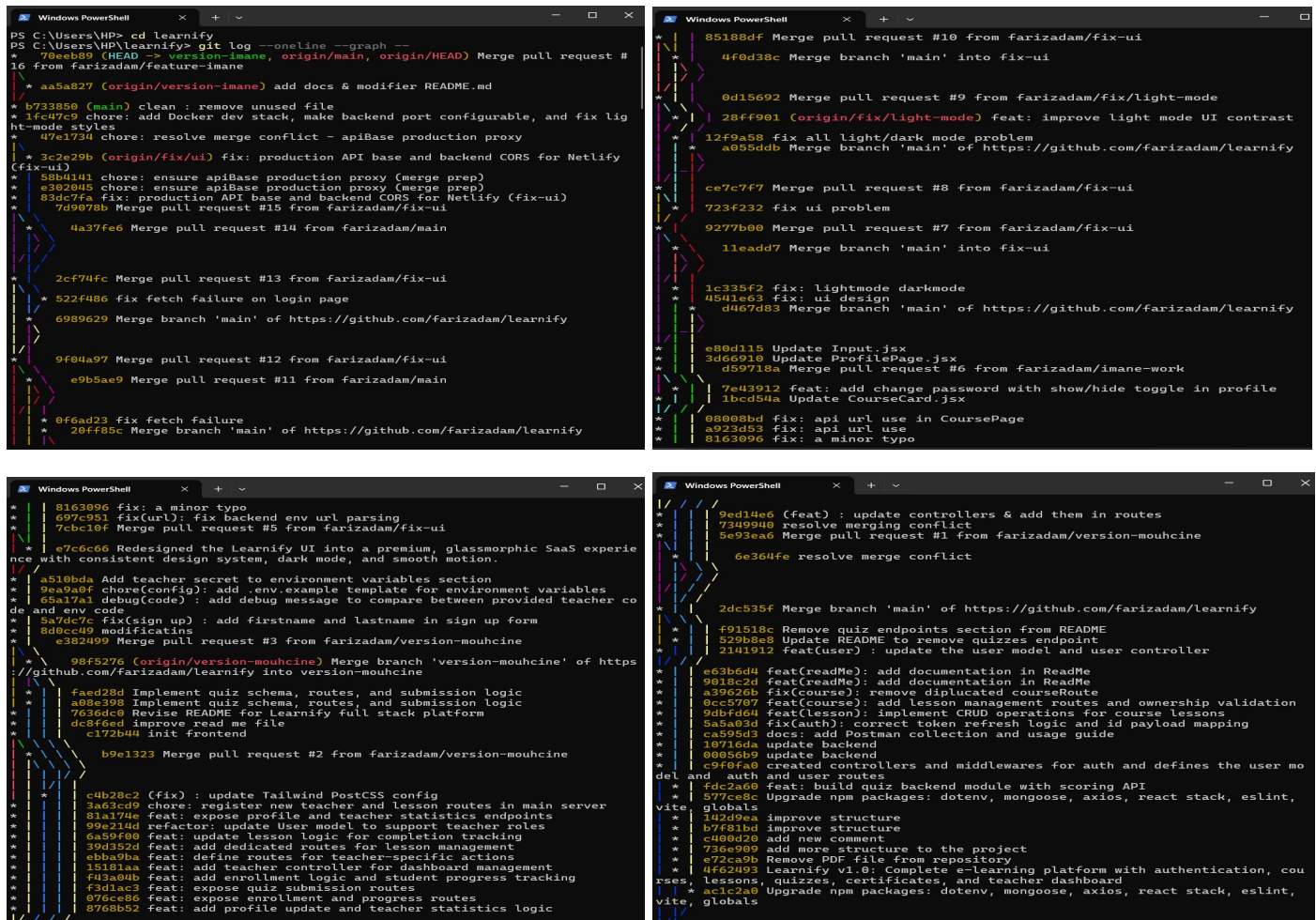
1.1 Convention de nommage et visibilité

Le dépôt est public, accessible à l'adresse : <https://github.com/farizadam/learnify>. Il contient l'intégralité du code source (frontend React + backend Node.js), la documentation et les fichiers de configuration.

Capture d'écran — Page d'accueil du dépôt GitHub :



Capture de git log --oneline --graph --all :



```

135, lessons, quizzes, certificates, and teacher dashboard
| | * ac1c2a0 Upgrade npm packages: dotenv, mongoose, axios, react stack, eslint,
vite, globals
| | /
| | /
| | /
* | c83f34f improve structure
* | 2acac23 improve structure
* | 48197ff add new comment
* | 5092c05 add more structure to the project
| | /
* | d209f5d (origin/version-adam) update client folder
* | 4ffb24e feat: initialize Vite React client and directory structure
* | 38c2745 fix: remove node_modules and .env from tracking and add gitignore
* | 4006066 feat: setup MERN directory structure and initial dependencies
* | 217b6c0 first commit
(END)

```

1.2 Arborescence du dépôt

learnify/

|— README.md

|— .gitignore

|— docker-compose.yml

|— docs/

|— client/

|— server/

Partie 2 — Stratégie des branches

Schéma des branches

Main

|— version-adam

|— version-mouhcine

|— version-imane

|— fix/ui

|— fix/light-mode

Tableau des commits par sprint

Branche	Rôle
main	Code de production stable
version-imane	Travail intégration (Imane)
version-adam	Développement backend (Adam)
version-mouhcine	Développement backend/quiz (Mohcine)
fix/ui	Corrections UI itératives
fix/light-mode	Correction dark/light mode

Partie 3 — Contributions individuelles

Membre	Commits	Lignes ajoutées	Modules principaux
FARIZ Adam	59 (adam + farizadam + Fariz Adam)	4315	Backend (server, routes, controllers, models, middleware, DB config), Docker, vite config
ALOUANI Mouhcine	36 (Alouani Mouhcine + mouh-cine47 + mouhcine)	6209	Frontend (QuizPage, Dashboard, CoursePage, Navbar, Sidebar, App.jsx, CSS, apiBase)
CHAMSSI Imane	10 (Imane321 + imane)	3795	Intégration frontend-backend, ProfilePage, CourseCard, corrections controllers

Capture GitHub Insights

```
PS C:\Users\HP> cd learnify
PS C:\Users\HP\learnify> git shortlog -sn --all
 47  adam
 22  Alouani Mouhcine
 12  mouh-cine47
  7  Fariz Adam
  5  Imane321
  5  farizadam
  5  imane
  2  mouhcine
PS C:\Users\HP\learnify> |
```

Partie 4 — Déploiement en ligne

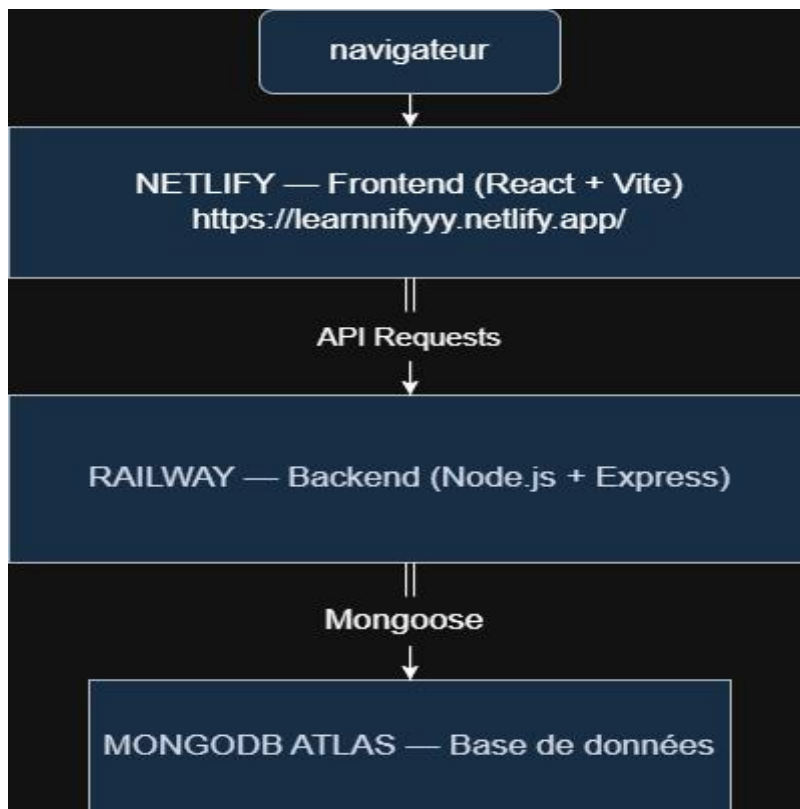
4.1 URLs publiques

<https://learnnifyyy.netlify.app/>

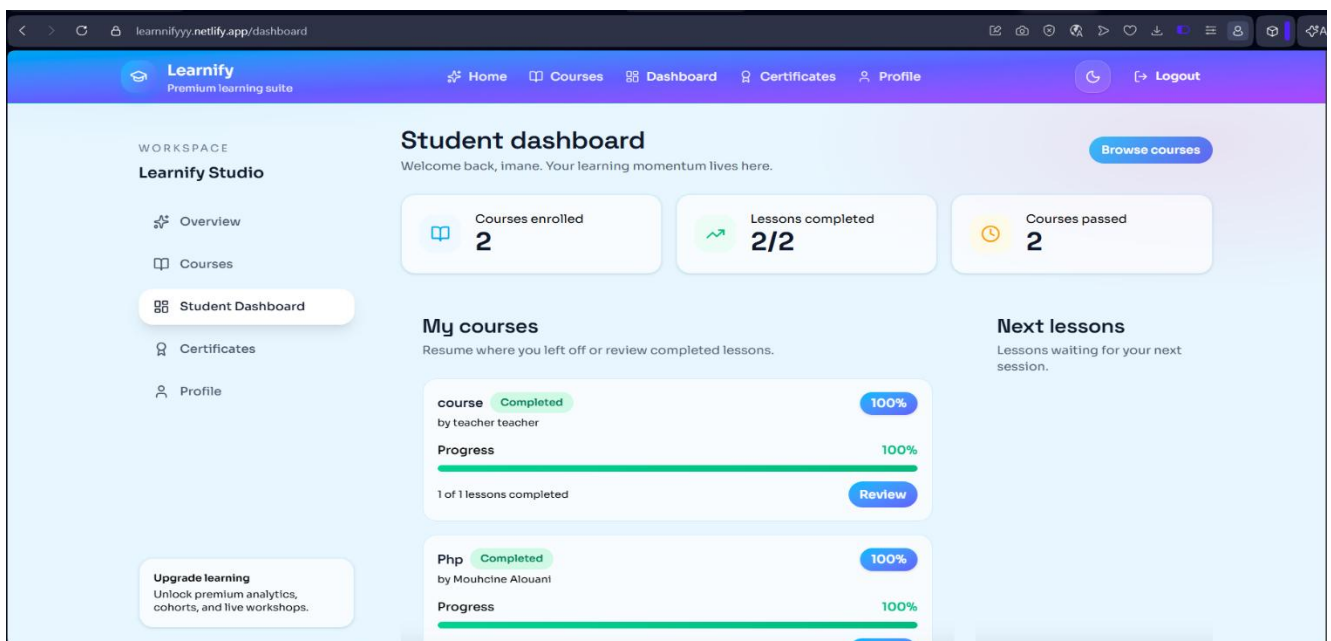
4.2 Identifiants de démonstration

Rôle	Email	Mot de passe
Étudiant	Sara.benmoussa@gmail.com	123456
Enseignant	Jalal.fathi@gmail.com	fathi1234

4.3 Diagramme de déploiement



Capture — Application ouverte dans le navigateur :



4.4 Solution de repli (si hors ligne le jour de la soutenance)

Le projet inclut une configuration Docker complète (docker-compose.yml + Dockerfile client + Dockerfile server). Lancement en local en 2 commandes :

```
git clone https://github.com/farizadam/learnify && cd learnify
```

```
docker-compose up --build
```

Partie 5 — Bilan technique

5.1 Difficultés rencontrées et solutions apportées

Difficulté	Solution apportée
La récupération simultanée des cours, de la progression et des scores de quiz a généré des requêtes lourdes avec plusieurs jointures, ce qui a ralenti considérablement l'affichage	Optimisation des performances via l'indexation et l'utilisation des pipelines d'agrégation MongoDB, permettant d'envoyer des données légères et structurées au frontend
Risque d'accès non autorisé aux routes sensibles (admin, validation de quiz) via la manipulation des requêtes HTTP ou de l'URL	Mise en place d'une authentification sécurisée basée sur JWT, stocké dans des cookies HttpOnly, et implémentation de middlewares de vérification des rôles (authentification, admin, enseignant)
Dark/Light mode — Incohérences visuelles entre les différents composants lors du changement de thème	Refonte complète du système de thème avec Tailwind CSS et mise en place d'un design system unifié pour assurer la cohérence de l'interface utilisateur
Problèmes liés à l'upload des fichiers en environnement de production	Adaptation du système de stockage pour l'environnement Railway, avec ajout de validations sur les types MIME et la taille des fichiers afin de garantir la sécurité et la stabilité.
Blocage de la communication entre le frontend (Netlify) et le backend (Railway) dû aux restrictions CORS	Configuration dynamique de l'URL de l'API via une variable d'environnement et autorisation explicite de l'origine du frontend dans le serveur backend

5.2 Dette technique identifiée

- Ajouter des tests unitaires (Jest pour le backend, Vitest pour le frontend) — couverture actuelle : 0%
- Mettre en place un pipeline CI/CD (GitHub Actions) pour les tests automatiques avant merge
- Migrer le stockage de fichiers vers Cloudinary ou AWS S3 pour les médias > 5 Mo
- Finaliser la fonctionnalité de correction dirigée (US-10) et les notifications email

Partie 6 — Rétrospective finale

Ce qui a bien marché :

La division des rôles (backend / frontend / intégration) a permis de travailler en parallèle sans blocages majeurs

Architecture MERN solide dès Sprint 1

Jira pour le suivi des tâches et la vélocité

Toutes les fonctionnalités Must livrées

Fonctionnalités Should également livrées

workflow Git collaboratif et traçable

Déploiement Netlify + Railway réussi

La conteneurisation Docker (docker-compose) permet de lancer tout l'environnement local en une seule commande

Ce qui n'a pas marché

Critères d'acceptation définis trop tard

Branches non alignées sur les sprints

SMTP non configuré en production

US-10 (correction dirigée) incomplète

Ce qu'on referait différemment

Nommer les branches sprint-1, sprint-2, develop, main

Rédiger tous les critères d'acceptation avant le début du développement

Créer des branches Git par User Story dès le départ pour éviter les conflits

Prévoir un Spike technique pour les fonctionnalités à risque avant le sprint

Configurer stockage cloud (Cloudinary)